Redis Cheat Sheet (Python / redis-py)

First 5 data types: String, List, Set, Sorted Set, Hash

Memorise the prefix: no prefix = String | **l / r** = List | **s** = Set | **z** = Sorted Set | **h** = Hash

Setup

```
import redis
r = redis.Redis(host="localhost", port=6379, decode_responses=True)
```

1. String : one value (cache, OTP, counter)

Function	Does
`r.set(k, v, ex=60)`	Save a value (optional expiry in seconds)
`r.get(k)`	Read it
`r.setnx(k, v)`	Set only if the key does not exist
`r.incr(k) / r.decr(k)`	+1 / -1
`r.incrby(k, n)`	Add n
`r.append(k, s)`	Add text to the end
`r.strlen(k)`	Length
`r.mset({...}) / r.mget([...])`	Set / get many
`r.ttl(k)`	Seconds left (-1 = never, -2 = gone)
`r.expire(k, s)`	Add an expiry
`r.persist(k)`	Remove the expiry
`r.exists(k) / r.delete(k)`	Check / delete

2. List : ordered line, duplicates OK (queue, latest N)

Function	Does
`r.rpush(k, *v)`	Add to the RIGHT end
`r.lpush(k, *v)`	Add to the LEFT end
`r.lrange(k, 0, -1)`	Read all (0 to -1 = first to last)
`r.llen(k)`	Length
`r.lindex(k, i)`	Item at position i
`r.lpop(k) / r.rpop(k)`	Remove and return from left / right
`r.blpop(k, timeout=5)`	Wait for an item (worker style)
`r.ltrim(k, 0, 4)`	Keep only the first 5
`r.lrem(k, 1, v)`	Remove 1 occurrence of v

Queue = rpush + lpop (first in, first out). **Stack** = lpush + lpop (last in, first out).

3. Set : unique, no order (visitors, tags, likes)

Function	Does
`r.sadd(k, *v)`	Add (duplicates ignored)
`r.smembers(k)`	Read all
`r.sismember(k, v)`	Is v inside? (True / False)
`r.scard(k)`	Count
`r.srem(k, v)`	Remove v
`r.spop(k)`	Remove and return a random item
`r.srandmember(k)`	Random item, keeps it
`r.sinter(a, b)`	In BOTH (mutual friends)
`r.sunion(a, b)`	In EITHER
`r.sdiff(a, b)`	In first, NOT in second

4. Sorted Set : unique + score, auto-sorted (leaderboard)

Function	Does
`r.zadd(k, {"Ravi": 500})`	Add with score
`r.zrange(k, 0, -1, withscores=True)`	Low to high
`r.zrevrange(k, 0, 2)`	High to low (top 3)
`r.zscore(k, m)`	Score of m
`r.zrank(k, m)`	Position, low to high (starts at 0)
`r.zrevrank(k, m)`	Position, high to low (leaderboard rank)
`r.zcard(k)`	Count
`r.zincrby(k, 100, m)`	Add to the score
`r.zrem(k, m)`	Remove
`r.zrangebyscore(k, 600, 800)`	Members with score in the range
`r.zcount(k, 600, 800)`	How many in the range

5. Hash : mini dictionary (profile, cart, session)

Function	Does
`r.hset(k, mapping={...})`	Set one or many fields
`r.hget(k, f)`	One field
`r.hmget(k, [f1, f2])`	Several fields
`r.hgetall(k)`	All fields and values
`r.hkeys(k) / r.hvals(k)`	Field names / values
`r.hlen(k)`	Number of fields
`r.hexists(k, f)`	Does the field exist?
`r.hdel(k, f)`	Delete a field
`r.hincrby(k, f, 1)`	Add to a number field

Works on ANY type

Function	Does
`r.type(k)`	Which type is this key?
`r.delete(k)`	Delete
`r.exists(k)`	Does it exist?
`r.expire(k, s) / r.ttl(k) / r.persist(k)`	Set / check / remove expiry
`r.scan_iter(match="user:*")`	Safely find keys (not KEYS, which blocks the server)

Memorise using pairs

Pattern	Functions
`add / read / remove`	String: set / get / delete | List: push / range / pop | Set: sadd / smembers / srem | ZSet: zadd / zrange / zrem | Hash: hset / hgetall / hdel
`count`	llen (List), scard (Set), zcard (ZSet), hlen (Hash)
`increase a number`	incr (String), zincrby (ZSet), hincrby (Hash)

Which type when?

I need...	Use
`OTP, cache, counter`	String
`Queue, latest N items`	List
`Unique likes, tags, mutual friends`	Set
`Leaderboard, top N`	Sorted Set
`User profile, cart, session`	Hash

One-line memory

String = one value | **List** = ordered line | **Set** = unique bag | **Sorted Set** = ranked unique bag | **Hash** = object with fields

Python differences from the CLI: zadd takes a dict; hset uses mapping=; blpop returns (key, value) or None; sismember returns True/False.